Gen AI Evaluation Project: "Build a Mini RAG-Powered Assistant"

Objective

Design and implement a basic Retrieval-Augmented Generation (RAG) assistant that can answer questions based on a custom document corpus. The assistant should use an LLM (like ChatGPT or any open-source model) and demonstrate understanding of Gen AI concepts, cloud deployment, and integration logic.

Project Requirements

1. Corpus Preparation

- Choose 3–5 documents (can be public PDFs, articles, or internal dummy docs).
- Preprocess and chunk the documents for embedding.

2. Embedding & Vector Store

- Use an embedding model (e.g., OpenAI, HuggingFace, or Azure OpenAI).
- Store embeddings in a vector database (e.g., FAISS, Pinecone, or any cloud-native option).

3. Query Handling

- Accept user queries via a simple Node.js frontend (can be CLI or basic web UI).
- Retrieve relevant chunks using similarity search.
- Pass context to an LLM for final response generation.

4. Cloud Deployment

- Deploy the solution on any cloud platform (Optional).
- Use GitHub for version control and optionally GitHub Copilot for coding assistance.

5. Documentation

- Include a README with architecture, setup instructions, and learnings.
- Highlight any challenges faced and how they were resolved.

Hackathon Use Case: Machine Learning Regression Model for Predictive Analytics

1. Problem Statement

Many industries rely on accurate numerical predictions to support decision-making. Whether forecasting sales, predicting customer spending, estimating delivery times, or determining equipment failure patterns, regression models play a central role. However, building an effective regression solution requires clean data, robust modeling, hyperparameter optimization, and clear evaluation metrics.

This use case focuses on developing a **Machine Learning Regression System** capable of predicting a continuous numeric value based on historical data. Participants will build, tune, and deploy a regression model that supports real-world prediction needs.

---

2. Objective

Build an end-to-end ML pipeline that:

- Ingests and preprocesses structured data
- Identifies correlations and relevant features
- Trains, optimizes, and evaluates a regression model
- Exposes the model through an API or simple UI
- Generates insights and visual explanations about predictions

---

3. Sample Use-Case Options

Participants may choose **one** domain to build the regression solution:

Option A: Sales Forecasting

Predict monthly/weekly/daily sales based on historical sales, promotions, seasonality, and product attributes.

Option B: House Price Prediction

Estimate property prices using features like location, size, rooms, amenities, and historical trends.

Option C: Customer Spend Prediction

Predict how much a customer will spend in the next month based on demographics, p
purchase behaviour, and engagement.

Option D: Delivery Time Estimation

Estimate delivery duration based on route, traffic, courier characteristics, and package
type.

---

4. Scope of the Solution

The solution will:

- Load dataset (CSV/Excel/Database)
- Perform cleaning, outlier removal, and feature engineering
- Train and compare multiple regression algorithms
- Produce evaluation metrics and visualizations
- Provide model inference via UI/API

Out of scope: - Real-time streaming predictions - Deep learning for large-scale time series
- Automated retraining pipelines

---

5. High-Level Architecture

5.1 Components

1. **Data Ingestion Layer**: Read structured files or connect to DB
2. **Data Preprocessing Module**:
   - Missing value handling
   - Outlier detection
   - Encoding & normalization
3. **Feature Engineering & Selection**
   - Correlation analysis
   - Domain-based feature creation
4. **Model Training & Optimization**
   - Linear Regression
   - Random Forest Regressor
   - Gradient Boosting / XGBoost / LightGBM
   - Hyperparameter tuning (GridSearch/Optuna)
5. **Evaluation Module**

- Error metrics (RMSE, MAE, R^2)
- Residual plots

6. **Deployment Layer**
   - REST API or simple UI
   - Containerization (Docker optional)

5.2 Architecture Diagram (Textual Representation)

```
Data → Preprocessing → Feature Engineering → Model Training → Evaluation → Deployment (API/UI)
```

---

6. Detailed Workflow

Step 1: Data Collection

- Read CSV or fetch dataset from database/URL
- Perform schema validation

Step 2: Data Preprocessing

- Missing value imputation
- Categorical encoding
- Scaling/normalization for numerical fields

Step 3: Exploratory Data Analysis (EDA)

- Distribution plots
- Correlation heatmaps
- Identify skewness and outliers

Step 4: Model Training

Train and compare multiple regression models: - **Baseline model:** Linear Regression - **Tree-based models:** Random Forest, XGBoost - **Regularization models:** Lasso, Ridge

Step 5: Hyperparameter Tuning

Use GridSearchCV / RandomizedSearchCV / Optuna to improve accuracy.

Step 6: Model Evaluation

Key metrics: - **RMSE (Root Mean Squared Error)** - **MAE (Mean Absolute Error)** - **R^2 Score**

Example output:

```
{
  "model": "RandomForestRegressor",
  "rmse": 12.4,
```

```
  "mae": 9.8,
  "r2": 0.91
}
```

Step 7: Deployment

Expose your final model using: - Flask API / FastAPI - Streamlit UI - Docker container (optional)

---

7. Tech Stack Options

Programming

- Python (Pandas, NumPy, Scikit-Learn)

Visualization

- Matplotlib, Seaborn, Plotly

Model Deployment

- Flask / FastAPI
- Streamlit
- Docker (optional)

Storage

- Local CSV
- SQLite/PostgreSQL

---

8. Evaluation Metrics

- Feature preparation accuracy (data quality)
- Model performance on metrics (MAE, RMSE, R^2)
- Interpretability (feature importance, plots)
- End-to-end pipeline automation
- UI/API usability

---

9. Challenges & Mitigations

Challenge	Mitigation
Missing or inconsistent data	Use imputation and validation rules
Overfitting on training data	Cross-validation and regularization
Highly correlated features	Apply PCA or remove redundant features

Challenge	Mitigation
Slow model training	Use optimized libraries like XGBoost

10. Deliverables

- ML notebook or script for full workflow
- Cleaned dataset used for training
- Trained regression model (.pkl)
- API/UI for predictions
- Documentation with architecture, results, and challenges

Hackathon Use Case: Neural Network-Based Classification System

1. Problem Statement

Classification is one of the most widely used applications of Machine Learning, enabling systems to categorize data into predefined classes. From spam detection, churn prediction, and image classification to fraud detection and medical diagnosis—classification models drive critical business decision-making.

Traditional ML models (Logistic Regression, Random Forest, SVM) work well for structured data but may struggle with large, high-dimensional, or complex patterns. Neural Networks (NNs) excel in such scenarios by learning complex, nonlinear relationships.

This use case focuses on building an end-to-end **Neural Network Classification System** using structured or unstructured data.

---

2. Objective

Build a complete classification pipeline using a neural network model that can:

- Process and transform input data
- Train a neural network classifier
- Tune hyperparameters for best performance
- Evaluate results using appropriate metrics
- Serve predictions through an API or simple UI

---

3. Sample Use-Case Options

Participants may choose **one** domain to apply NN-based classification:

Option A: Customer Churn Prediction

Predict whether a customer will churn based on usage patterns, demographics, and engagement.

Option B: Email Spam Detection

Classify emails into *spam* or *not spam* based on text features.

Option C: Employee Attrition Prediction

Predict if an employee is likely to leave.

Option D: Image Classification (Simple CNN)

Identify objects (e.g., cats vs. dogs) using a small Convolutional Neural Network.

Option E: Loan Approval Classification

Classify applicants as *approved* or *not approved* based on financial and demographic data.

---

4. Scope of the Solution

The solution will:

- Load and preprocess structured or image data
- Encode labels and features
- Build a neural network model
- Train, validate, and tune hyperparameters
- Generate classification metrics
- Deploy model with an API/UI

Out of scope: - Large-scale deep learning (e.g., full-fledged vision transformers) - Real-time model drift monitoring - Distributed training

---

5. High-Level Architecture

5.1 Components

1. **Data Ingestion Layer**: CSV upload or image dataset import
2. **Data Preprocessing Module**:
   - Encoding categorical variables
   - Tokenization (for text)
   - Normalization/scaling
3. **Neural Network Design**
   - Dense Neural Network (DNN) for structured data
   - CNN for image-based classification
4. **Training & Optimization**
   - Backpropagation
   - Early stopping

- Dropout layers to reduce overfitting

5. **Evaluation Module**
   - Accuracy, Precision, Recall, F1-score
   - ROC curve & confusion matrix
6. **Deployment Layer**
   - REST API or UI

5.2 Architecture Diagram (Textual Representation)

```
Data → Preprocessing → Neural Network Model → Evaluation → Deployment (API/UI)
```

---

6. Detailed Workflow

Step 1: Data Understanding & Preparation

- Validate dataset quality
- Identify class imbalance
- Split into train–validation–test sets

Step 2: Preprocessing

- One-hot encoding for categorical fields
- Tokenization & embedding (if text)
- Image resizing (if images)
- StandardScaler/MinMaxScaler for numeric features

Step 3: Neural Network Architecture

For structured data:

```
Input Layer → Dense Layers → Dropout → Output Layer (Softmax/Sigmoid)
```

For images:

```
Conv2D → MaxPooling → Flatten → Dense → Output Layer
```

Step 4: Model Training

- Optimizers: Adam, SGD
- Loss functions: Binary Crossentropy / Categorical Crossentropy
- Batch training with 20–50 epochs

Step 5: Hyperparameter Tuning

Tune: - Learning rate - Number of layers & neurons - Activation functions (ReLU, tanh) - Dropout rate

Step 6: Evaluation

Example output:

```
{
  "accuracy": 0.94,
  "precision": 0.91,
  "recall": 0.89,
  "f1_score": 0.90
}
```

Visual outputs: - Confusion Matrix - ROC Curve - Feature importance (SHAP) for explainability

Step 7: Deployment

Expose predictions using: - FastAPI/Flask - Streamlit UI - Docker container (optiona

---

7. Tech Stack Options

Frameworks

- TensorFlow / Keras
- PyTorch

Libraries

- Scikit-Learn
- Pandas, NumPy
- Matplotlib/Seaborn

Deployment

- Flask / FastAPI
- Streamlit

---

8. Evaluation Metrics

- Accuracy
- Precision & Recall
- F1-Score
- AUC-ROC Score
- Training vs. validation loss curves

9. Challenges & Mitigations

Challenge	Mitigation
Class imbalance	SMOTE, focal loss, class weighting
Overfitting	Dropout, early stopping
Long training time	GPU acceleration, reduce batch size
Poor feature representation (text/images)	Use embeddings or CNN architectures

10. Deliverables

- Full NN training notebook
- Cleaned dataset
- Trained NN model (.h5 / .pt)
- Evaluation report with metrics & visualizations
- API/UI for inference
- Documentation with architecture & troubleshooting